Pulsos optimizados con GRAPE

Genesis Silesky

21 de octubre de 2026

1. Pulsos optimizados con GRAPE: control como problema de optimización

1.1. Objetivo

Diseñar con un algoritmo numérico (GRAPE) la forma de un pulso de radiofrecuencia que prepare un estado de Bell en dos espines acoplados, y comprobar cómo se comporta frente a errores de frecuencia y de amplitud.

1.2. Conceptos previos

Concepto previo: control con Hamiltoniano, fidelidad y gradiente

Ver capítulo 12 (control con pulsos y acoplamiento), capítulo 13 (fidelidad) y capítulo 19 (optimización con gradiente). La *fidelidad* con un estado objetivo puro $|\phi\rangle$ es $F = |\langle\phi|\psi\rangle|^2$ (vale 1 si son iguales).

1.3. Fundamento teórico

Hasta ahora los pulsos tenían forma rectangular (*pulsos duros*: amplitud constante durante cierto tiempo). Pero la forma de un pulso se puede elegir. Según las notas del curso, los *pulsos con forma* (gaussianos, de Hermite) reducen los errores de excitación fuera de banda, y los algoritmos de optimización numérica como **GRAPE** y **SMP** ajustan la forma de los pulsos de control para maximizar la fidelidad de una operación dada: tratan el control como un problema de optimización, no de diseño manual. La guía de la práctica pide preparar un estado de Bell con GRAPE y hacer el pulso resistente a errores de ± 10 Hz en la frecuencia y ± 5 % en la amplitud.

El problema. Dos espines con acoplamiento $J = 696$ Hz y control de radiofrecuencia en x e y sobre cada uno:

$$H(t) = \frac{2\pi J}{4}\sigma_z \otimes \sigma_z + \frac{2\pi}{2}\sum_{j=1}^{4} u_j(t)\,\sigma_j, \tag{1}$$

con cuatro controles $u_j(t)$ (en Hz): σ_x y σ_y de cada espín. Se busca la forma de $u_j(t)$ que lleve $|00\rangle$ a $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$.

GRAPE. (*Gradient ascent pulse engineering.*) La duración total T se divide en N pasos de amplitud constante, y la amplitud de cada control en cada paso es un parámetro. Se maximiza

$$F = \left|\langle\Phi^+|\,U_N \cdots U_2 U_1\,|00\rangle\,\right|^2, \qquad U_k = e^{-iH_k\,\Delta t}. \tag{2}$$

El gradiente respecto de cada amplitud se obtiene propagando el estado hacia adelante y el objetivo hacia atrás (en primera aproximación en Δt), y un optimizador (aquí L-BFGS-B) lo usa para mejorar los parámetros.

Robustez. Para que el pulso resista errores se optimiza el *promedio* de la fidelidad sobre varias condiciones de error a la vez: desintonías $\pm\Delta$ en ambos espines y factores de escala de amplitud.

Fuentes y alcance

Este capítulo se basa en el resumen de las notas (qué son GRAPE y SMP) y en la guía de la práctica. Los archivos de teoría de pulsos con forma y de optimización numérica son escaneos sin texto legible. Por eso aquí **solo se implementa GRAPE** con el estado de Bell; no se implementan SMP, los pulsos duros de comparación ni los pulsos gaussianos o de Hermite.

1.4. Código (completamente comentado)

```
# %% [markdown]
# # 23 - Pulsos optimizados con GRAPE: preparar un estado de Bell con pulsos de forma libre

# %% Celda 1: importaciones
import numpy as np
from scipy.linalg import expm
from scipy.optimize import minimize

I2 = np.eye(2, dtype=complex)
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)

# %% Celda 2: el problema de control
# Dos espines acoplados (J = 696 Hz, como en el capitulo 12). Se controlan con
# campos de radiofrecuencia en x e y sobre cada espin; la amplitud de cada campo
# (en Hz) puede cambiar en el tiempo. El Hamiltoniano es
#     H(t) = 2*pi*J*(sz (x) sz)/4  +  2*pi * suma_j  u_j(t) * sigma_j / 2,
# con cuatro controles: sx y sy en cada espin. Se busca la forma de u_j(t) que lleva
# |00> al estado de Bell (|00> + |11>)/sqrt2.
J = 696.0
H_deriva = 2 * np.pi * J * np.kron(sz, sz) / 4
OPERADORES = [np.kron(sx, I2), np.kron(sy, I2), np.kron(I2, sx), np.kron(I2, sy)]
H_controles = [2 * np.pi * op / 2 for op in OPERADORES]

T = 1.0e-3                          # duracion total del pulso: 1 ms
N = 20                              # numero de pasos de amplitud constante
dt = T / N
U_MAX = 5000.0                      # amplitud maxima de cada control: 5 kHz

inicial = np.array([1, 0, 0, 0], dtype=complex)                  # |00>
objetivo = np.array([1, 0, 0, 1], dtype=complex) / np.sqrt(2)    # Bell


def H_con_errores(desint=0.0, escala=1.0):
    """Deriva con un error de frecuencia (desint, en Hz) en ambos espines; los controles se
    ↪ escalan por 'escala'."""
    error = 2 * np.pi * desint * (np.kron(sz, I2) + np.kron(I2, sz)) / 2
    return H_deriva + error, escala


# %% Celda 3: GRAPE
# GRAPE (gradient ascent pulse engineering) trata el pulso como un problema de
# optimizacion: la forma del pulso se divide en N pasos y la amplitud de cada
# control en cada paso es un parametro. Se maximiza la fidelidad
#     F = |<objetivo| U(T) |inicial>|^2.
# El gradiente respecto de cada amplitud se obtiene propagando el estado hacia
# adelante y el objetivo hacia atras (aproximacion de primer orden en dt).
```

```python
def propagadores(u, H0, escala):
    return [expm(-1j * (H0 + escala * sum(u[k, j] * H_controles[j] for j in range(4))) * dt) for
    ↪  k in range(N)]


def fidelidad_y_gradiente(u, H0, escala):
    U = propagadores(u, H0, escala)
    adelante = [inicial]
    for Uk in U:
        adelante.append(Uk @ adelante[-1])
    atras = [None] * (N + 1)
    atras[N] = objetivo
    for k in range(N - 1, -1, -1):
        atras[k] = U[k].conj().T @ atras[k + 1]
    c = np.vdot(objetivo, adelante[N])
    grad = np.zeros_like(u)
    for k in range(N):
        for j in range(4):
            dc = np.vdot(atras[k + 1], (-1j * dt * escala * H_controles[j]) @ adelante[k + 1])
            grad[k, j] = 2 * np.real(np.conj(c) * dc)
    return abs(c) ** 2, grad


def optimizar(errores, semilla=0, intentos=4):
    """Maximiza la fidelidad PROMEDIO sobre una lista de errores (desintonia, escala)."""
    mejor = None
    for s in range(intentos):
        rng = np.random.default_rng(semilla + s)
        u0 = rng.uniform(-1000, 1000, (N, 4))

        def costo(x):
            u = x.reshape(N, 4)
            F, G = 0.0, np.zeros_like(u)
            for desint, escala in errores:
                H0, esc = H_con_errores(desint, escala)
                f, g = fidelidad_y_gradiente(u, H0, esc)
                F += f / len(errores)
                G += g / len(errores)
            return -F, -G.ravel()

        res = minimize(costo, u0.ravel(), jac=True, method="L-BFGS-B",
                       bounds=[(-U_MAX, U_MAX)] * (N * 4), options={"maxiter": 400})
        if mejor is None or res.fun < mejor.fun:
            mejor = res
    return mejor.x.reshape(N, 4), -mejor.fun


# %% Celda 4: pulso optimizado para el caso ideal (sin errores)
u_ideal, F_ideal = optimizar([(0.0, 1.0)])
print(f"Fidelidad del pulso GRAPE (caso ideal): {F_ideal:.4f}")
print("Amplitud maxima usada:", round(float(np.abs(u_ideal).max())), "Hz (limite", int(U_MAX), "
    ↪ Hz)")

# %% Celda 5: ese pulso frente a errores
# En un equipo real la frecuencia del pulso puede estar algo desafinada y la
# amplitud puede no ser exacta. Se evalua el pulso con desintonias de +-10 Hz
# y errores de amplitud de +-5 %.
CASOS = [(d, e) for d in (-10.0, 0.0, 10.0) for e in (0.95, 1.0, 1.05)]


def evaluar(u):
    fid = [fidelidad_y_gradiente(u, *H_con_errores(d, e))[0] for d, e in CASOS]
    return np.array(fid)
```

```python


f1 = evaluar(u_ideal)
print(f"Sin robustez: fidelidad promedio = {f1.mean():.4f}, minima = {f1.min():.4f}")

# %% Celda 6: pulso robusto
# Para hacer el pulso resistente, se optimiza el PROMEDIO de la fidelidad sobre
# todos los errores a la vez (9 combinaciones de desintonia y amplitud).
u_robusto, _ = optimizar(CASOS)
f2 = evaluar(u_robusto)
print(f"Con robustez: fidelidad promedio = {f2.mean():.4f}, minima = {f2.min():.4f}")
print(f"En el caso ideal (sin errores) el pulso robusto da {evaluar(u_robusto)[CASOS.index((0.0, 1.0))]:.4f}")

# %% Celda 7: errores mayores
# Con los errores del experimento (+-10 Hz, +-5 %) el pulso sin robustez ya se
# comporta bien, porque el pulso dura solo 1 ms. Con errores mas grandes la
# diferencia se nota: desintonias de +-300 Hz y errores de amplitud de +-20 %.
CASOS_GRANDES = [(d, e) for d in (-300.0, 0.0, 300.0) for e in (0.8, 1.0, 1.2)]


def evaluar_grandes(u):
    return np.array([fidelidad_y_gradiente(u, *H_con_errores(d, e))[0] for d, e in CASOS_GRANDES])


u_robusto_grande, _ = optimizar(CASOS_GRANDES)
for nombre, u in (("sin robustez", u_ideal), ("robusto (errores grandes)", u_robusto_grande)):
    f = evaluar_grandes(u)
    print(f"{nombre:26s}: fidelidad promedio = {f.mean():.4f}, minima = {f.min():.4f}")
```

1.5. Explicación celda por celda

Celda 2 Define H con deriva, controles, $T = 1$ ms, $N = 20$ pasos, amplitud máxima de 5 kHz y el estado objetivo.

Celda 3 GRAPE: propagadores, fidelidad, gradiente y la función `optimizar`, con cuatro intentos desde puntos iniciales distintos.

Celda 4 Optimiza el pulso del caso ideal (sin errores).

Celda 5 Evalúa ese pulso con desintonías de ± 10 Hz y errores de amplitud de ± 5 % (nueve combinaciones).

Celda 6 Optimiza el promedio sobre esas nueve combinaciones.

Celda 7 Repite con errores mayores (± 300 Hz y ± 20 %).

1.6. Salida obtenida

```
Fidelidad del pulso GRAPE (caso ideal): 0.9979
Amplitud maxima usada: 1694 Hz (limite 5000 Hz)
Sin robustez: fidelidad promedio = 0.9939, minima = 0.9889
Con robustez: fidelidad promedio = 0.9927, minima = 0.9902
En el caso ideal (sin errores) el pulso robusto da 0.9966
sin robustez              : fidelidad promedio = 0.4130, minima = 0.0809
robusto (errores grandes) : fidelidad promedio = 0.9520, minima = 0.8751
```

Es determinista (semillas fijas).

1.7. Interpretación

El pulso GRAPE prepara el estado de Bell con fidelidad 0.9979, usando como máximo 1694 Hz de amplitud. Con los errores de la guía (± 10 Hz y ± 5 %), el pulso sin robustez ya mantiene una fidelidad promedio de 0.9939 (mínima 0.9889), y el robusto 0.9927 (mínima 0.9902): **casi no hay diferencia**, porque el pulso dura solo 1 ms y esos errores son pequeños. Con errores mayores (± 300 Hz y ± 20 %) la diferencia es grande: la fidelidad promedio del pulso sin robustez cae a 0.4130 (mínima 0.0809), mientras que el pulso optimizado para esos errores mantiene 0.9520 (mínima 0.8751). Es el costo típico: el pulso robusto es un poco menos preciso en el caso ideal (0.9966 frente a 0.9979).

1.8. Ejercicios

1. Aumenta T a 2 ms y mira cómo cambia la fidelidad y la robustez.
2. Reduce la amplitud máxima a 1 kHz: ¿se sigue alcanzando el Bell?
3. Cambia el estado objetivo a $|\Psi^+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$.

1.9. Errores comunes

- Creer que el optimizador siempre halla el máximo global: GRAPE puede quedarse en un máximo local, por eso se prueban varios puntos iniciales.
- Olvidar que la fidelidad reportada se calcula con el propagador exacto, mientras que el gradiente usa una aproximación de primer orden.